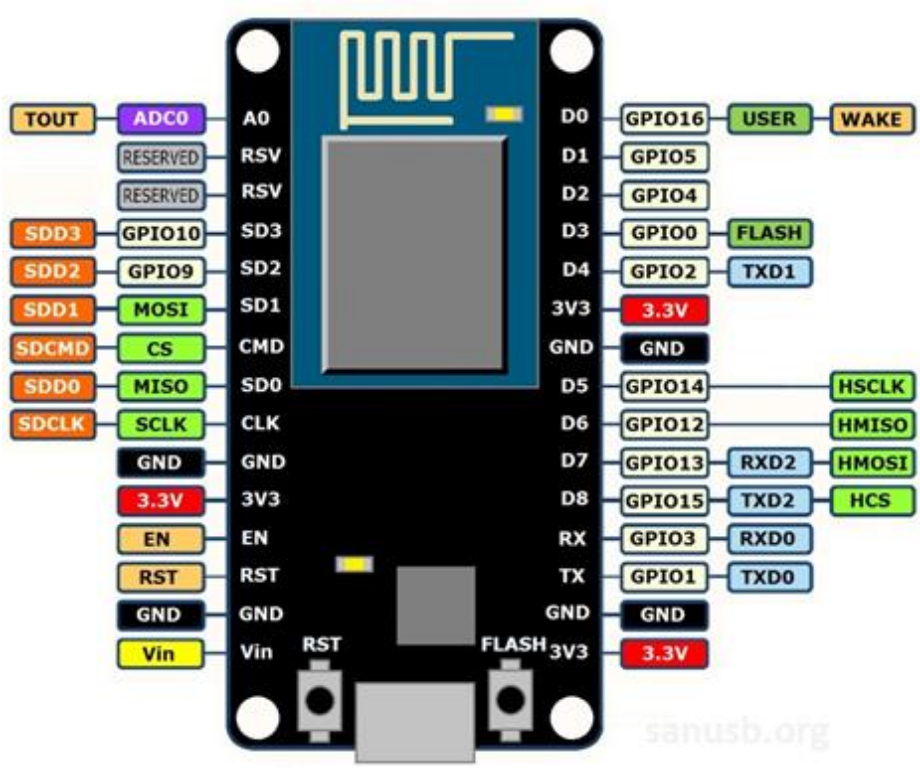
Aplicações Práticas de ESP8266 em IoT



Grupo SanUSB

INTRODUÇÃO

O NodeMCU ESP-12E é uma versão integrada do popular ESP8266, um *Serial to Wi-Fi System On a Chip (SoC)*, que apareceu pela primeira vez em 2013 e lançado no mercado já no ano seguinte. O ESP8266 foi desenvolvido pela empresa chinesa com sede em Shangai, *Espressif Systems*, uma fabricante de circuitos integrados focada no desenvolvimento de chips de RF, particularmente WiFi.

Os dois módulos mais conhecidos dessa família atualmente são a ESP-01 e o ESP-12E. O ESP-01 tem sido amplamente utilizado em projetos de IoT devido ao tamanho e custo, porém possui apenas 2 pinos digitais disponíveis. Por outro lado, o NodeMCU ESP-12E surge com mais pinos e periféricos. Algumas características do NodeMCU são descritas abaixo:

- Suporte a clientes TCP em múltiplos canais (máx. 5);
- Pinos GPIO (D0 a D8 e SD1 a SD3), PWM (D1 a D8), I2C, SPI e sensor interno de temperatura;
- Um conversor AD de 10 bits em AD0;
- Tensão de entrada Vin: 4.5V a 9V, e alimentação por USB;
- Corrente de trabalho: ≈70mA (máx. 200mA), *standby* menor que 200uA;
- Taxa de transmissão de dados: 110 a 460800bps;
- Suporte a atualização remota de firmware (OTA);
- Temperatura de trabalho:-40℃ a ＋125℃ ;
- Drive com dupla ponte H;
- System-On-Chip com Wi-Fi embutido;
- CPU que opera em 80MHz, com possibilidade de operar em 160MHz;
- Arquitetura RISC de 32 bits;
- 32KBytes de RAM para instruções;
- 96KBytes de RAM para dados;
- 64KBytes de ROM para boot;
- Possui uma memória Flash SPI externa de 4Mbytes;

Existem módulos de diferentes tamanhos e fabricantes. Até o presente momento, existem módulos numerados de ESP-01 até ESP-12. O objetivo dos modelos ESP-01 e

ESP-10 era servir como gateway Serial-WiFi, ou seja, de um lado eles recebem comandos AT via Serial (UART) e interagem com a rede WiFi por meio de conexões TCP/UDP. Os demais também podem operar nesse modo, embora sejam capazes de desempenhar mais funcionalidades, ou seja, como microcontroladores com Wi-Fi.

O módulo Wifi ESP8266 NodeMCU ESP-12E é uma das placas mais interessantes da família ESP8266, já que pode ser facilmente ligada à um computador e programada utilizando a IDE do Arduino, pela IDE platformIO, entre outras formas como por comandos AT ou com a linguagem Lua descritos no final do trabalho.

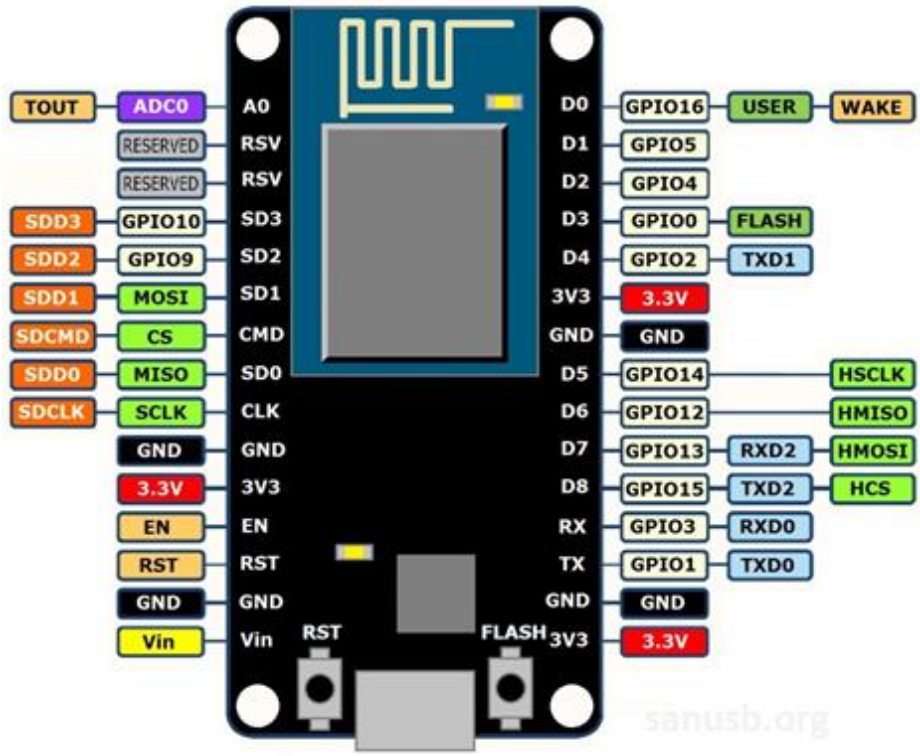


Figura 1: Placa ESP8266 NodeMCU

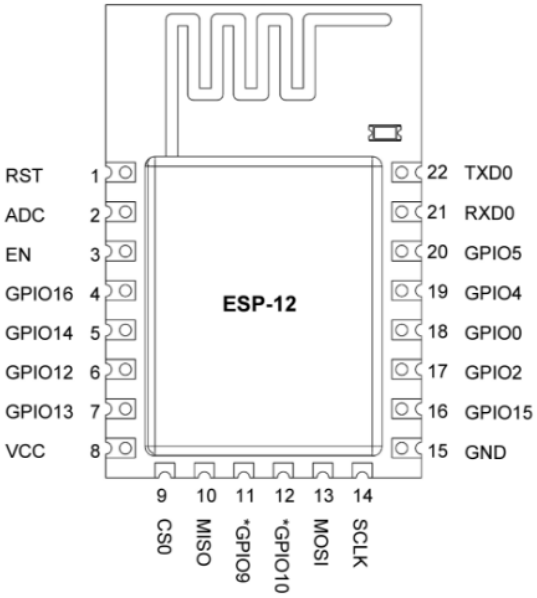
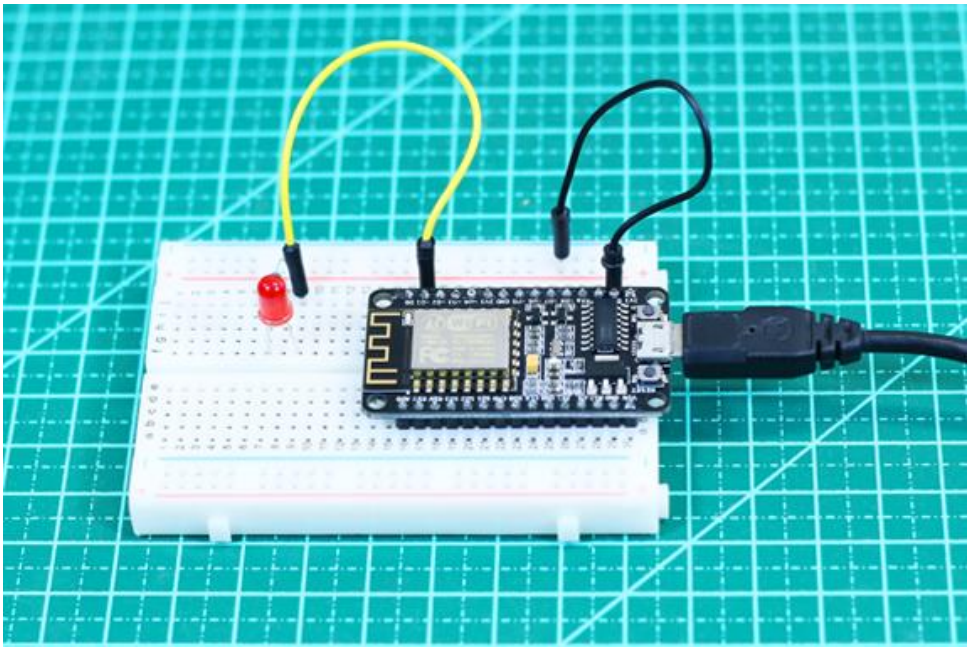


Figura 2: Chip ESP8266



Essa placa possui suporta funções como PWM, I2C e 1-wire. Tem antena embutida, conversor USB-TLL integrado e o seu formato é ideal para ambientes de prototipação, encaixando facilmente em um *protoboard*.

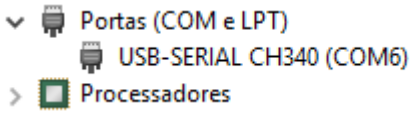
1.1 Hardware do módulo ESP8266

O módulo Wifi ESP8266 NodeMCU tem dois botões, conforme mostrado na imagem abaixo: **Flash** (utilizado na gravação do firmware) e **RST** (Reset). No mesmo

lado tem-se o conector USB para alimentação e conexão com o computador. O Led azul é ligado ao pino Tx que também é pino de GPIO 1.

No lado oposto, temos o ESP-12E e sua antena embutida, já soldado na placa. Nas laterais temos os pinos de GPIO, alimentação externa, comunicação, etc.

No Windows, a instalação e download de drivers é normalmente realizada de forma automática. Para verificar a instalação do adaptador USB-serial, acesse o Gerenciador de dispositivos clicando **Win + x**. Caso não esteja instalado, baixe e instale os drivers USB-serial em http://sanusb.org/esp/driver. Após a instalação, um dispositivo USB-TTL CH340 (NodeMCU 0.9) ou USB-TTL CP210 (NodeMCU 1.0) será adicionado ao gerenciador de dispositivos. Nesse exemplo, foi atribuída a porta COM6:



Vamos programar o módulo utilizando o Lua, e isso pode ser feito, à princípio, de duas maneiras.

A plataforma NodeMCU foi iniciada em 2014, menos de um ano após o começo das vendas do ESP8266, e contendo uma série de recursos adicionais para uso embarcado, e uma placa de desenvolvimento (de uso opcional) baseada no popular módulo ESP-12.

A imagem acima mostra um dos modelos do módulo de desenvolvimento do NodeMCU – note que na sua parte superior está presente um módulo ESP-12 completo. Além de apresentar os pinos do ESP-12 em uma configuração fácil de plugar em protoboards ou outros recursos de prototipação, o módulo de desenvolvimento também oferece, entre outros confortos, a regulação de tensão, uma porta USB para a comunicação USB-serial, um botão de reset e um botão '*flash*', cujo pressionamento coloca o ESP8266 no modo de *upload* de firmware.

Os módulos de desenvolvimento NodeMCU são didáticos, mas é possível utilizar também um ESP8266 12E comum.

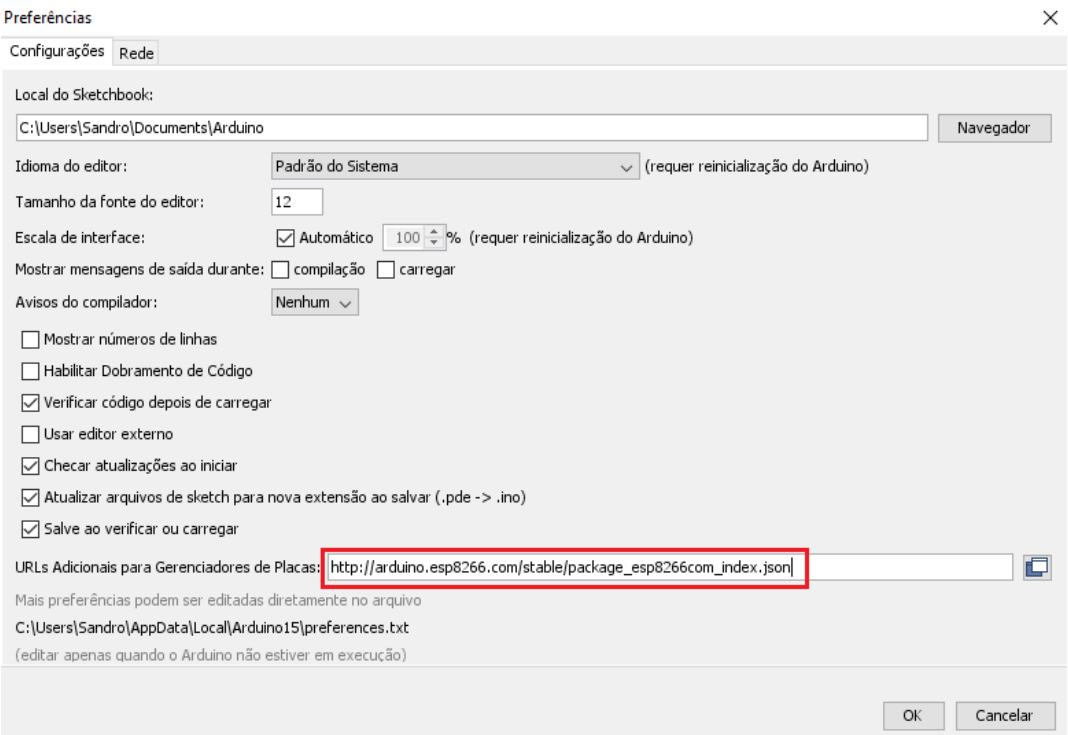
Dessa forma, o uso desse hardware é opcional. Tendo alguns jumpers e um adaptador USB-serial, é possível instalar o firmware NodeMCU na maioria dos módulos ESP8266, desde os modestos ESP01 até os avançados como o ESP12 e a cada vez maior gama de derivados e placas de desenvolvimento.

Integrando o ESP8266 à IDE Arduino

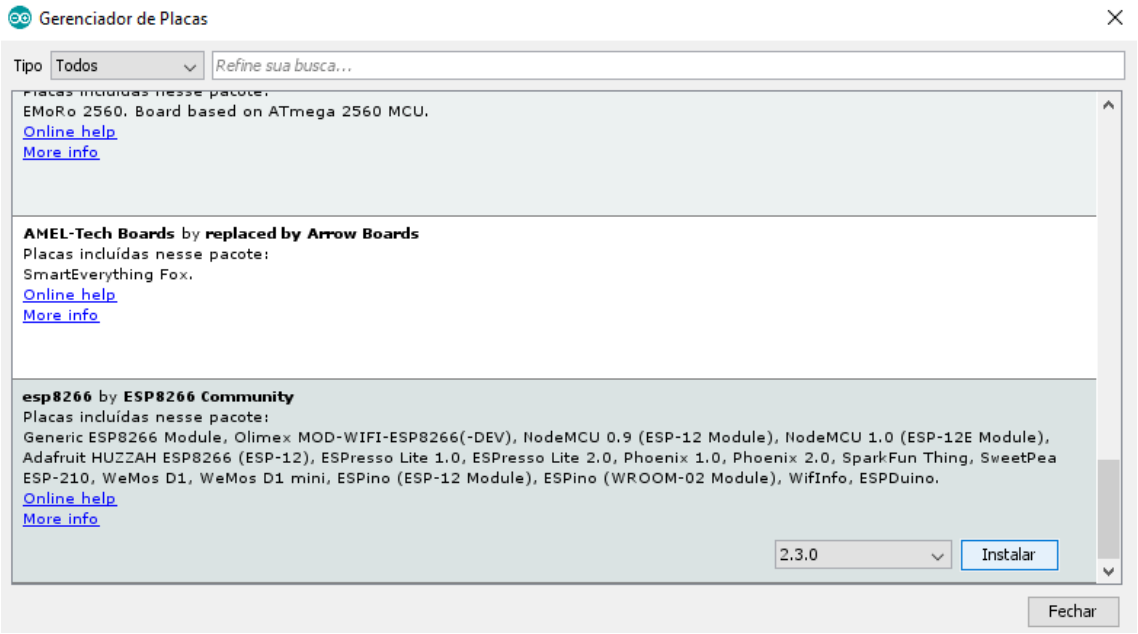
Para programar e usar o NodeMCU como se fosse um Arduino, a boa notícia é que é possível escrever-se firmwares personalizados e carregá-los no chip ("flash-it"). É importante lembrar que qualquer novo "firmware personalizado" irá substituir qualquer coisa previamente armazenada na memória flash do chip, incluindo o firmware original carregado em fábrica (aquele que aceita os comandos AT). Embora possamos usar o SDK do fabricante para o desenvolvimento de firmwares personalizados, é mais simples utilizar nesse caso a IDE Arduino. A IDE do Arduino torna muito mais fácil a programação do ESP8266, mantendo os comandos compatíveis com Arduino, basta fazer um #include das bibliotecas, sem se preocupar com detalhes do SDK da ESPRESSIF. Bibliotecas em https://github.com/esp8266/Arduino/tree/master/libraries.

Na IDE Arduino, abra a janela de preferências em Arquivo e digite a URL (marcado em vermelho na foto abaixo) no campo URLs adicionais para Gerenciadores de Placas e selecione OK.

http://arduino.esp8266.com/stable/package_esp8266com_index.json

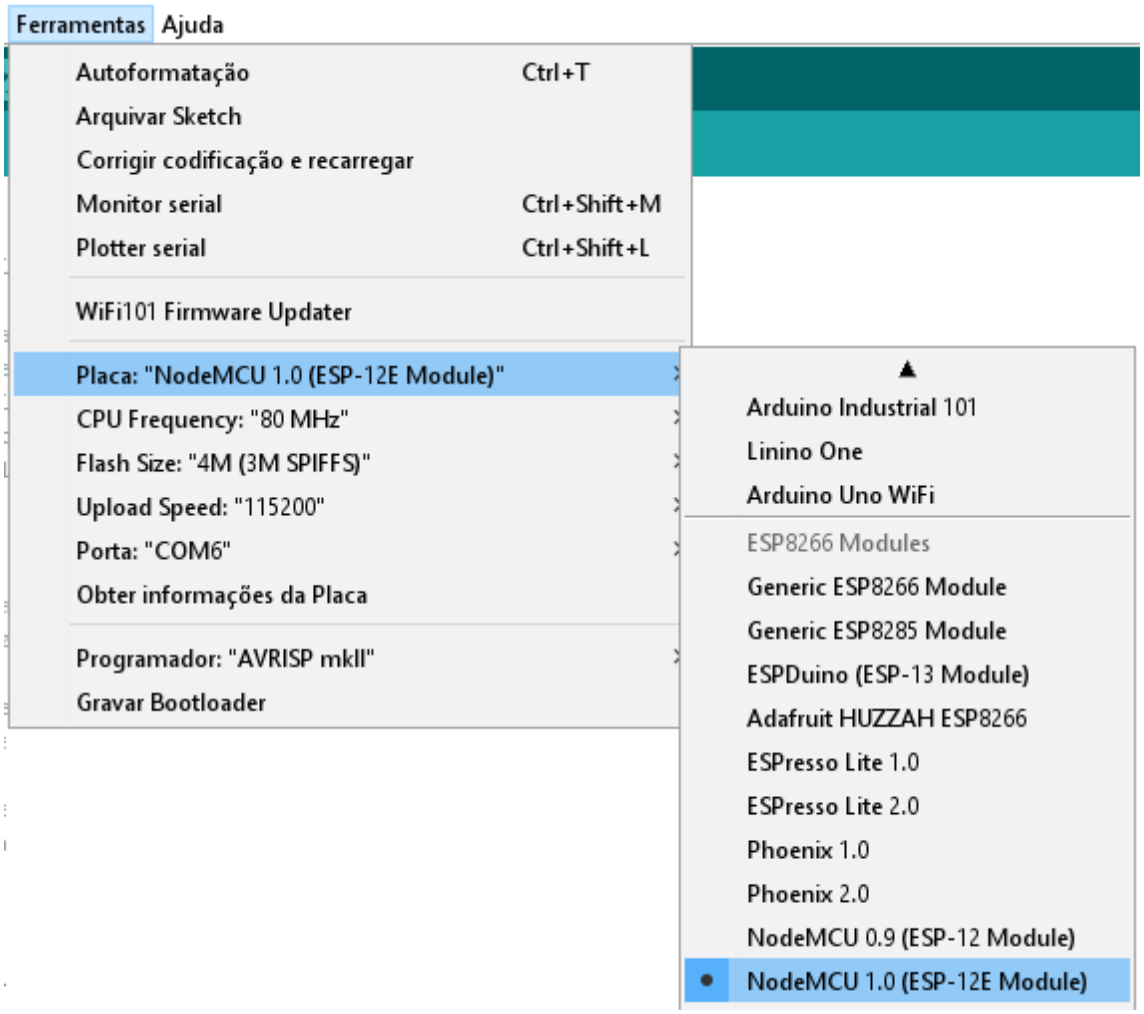


Selecione **MENU → Ferramentas → Placas → Gerenciador de placas**…e vá rolando até encontrar a opção: *esp8266 by ESP8266 Community* , a qual deverá ser o último item da lista e clique **INSTALAR.** É importante salientar que todas as blibliotecas instaladas ficam disponíveis em um diretório convencional como em ~\Documents\Arduino\Libraries.

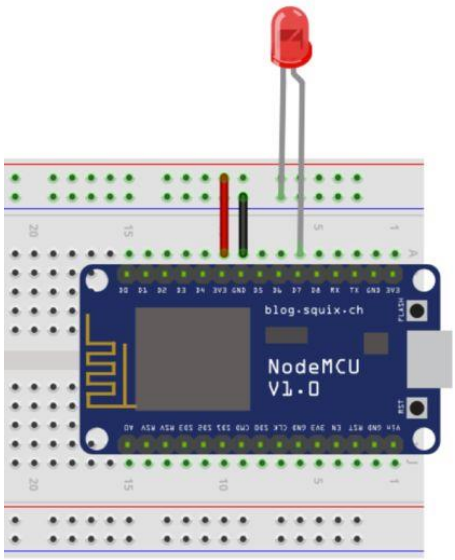


A principal diferença entre o NodeMCU 0.9 e NodeMCU 1.0, é que a versão do conversor USB-*Serial* do NodeMCU 0.9 é o chip CH340 e do NodeMCU 0.9 é o *chip CP2102*, para o qual deveremos instalar o driver *Virtual COM Port (VCP)*. É possível encontrar o driver apropriado CP2102 para o computador no seguinte link: sanusb.org/esp/driver.

Depois de restartar a IDE Arduino, é possível selecionar a placa no menu: **Tools → Ferramentas → Placa → NodeMCU 1.0 (ESP-12E Module).** Em seguida, especificar a correta frequência de operação da CPU: (**Ferramentas → CPU Frequency: “” → 80MHz)** e velocidade de comunicação (**Ferramentas → Upload Speed: “” → 115,200**). Finalmente, selecionar o port apropriado ao seu computador: (**Ferramentas→ Porta → /COM6**).



Neste ponto estamos prontos para escrever nosso próprio firmware e enviá-lo ao dispositivo, mas vamos primeiramente tentar um dos exemplos incluídos com a biblioteca: **Arquivo → Exemplos → ESP8266WiFi → Blink**. Após o *upload*, podemos abrir a janela do *Serial Monitor* e observar os resultados. Verifique que 115.200 baud é a velocidade selecionada no menu do canto inferior direito do *Serial Monitor*.



Você poderá testar o código usando tanto “D7” quanto “13”. ambas formas funcionarão. O pino D7 não precisa de um resistor externo para alimentar o LED, pois possui um internamente. Abaixo um código simples para piscar o LED:

```
/***********************************************
  Blink
  Connected to pin D7 (GPIO13) ESP8266 NODEMCU
 ***********************************************/
#define ledPin 13  // #define ledPin D7
void setup()
{
  pinMode(ledPin, OUTPUT);
}
void loop()
{
  digitalWrite(ledPin, HIGH);
  delay(1000);
  digitalWrite(ledPin, LOW);
  delay(1000);
}
```

Há uma relação entre alguns dos pinos do NodeMCU e do Arduino, automaticamante identificados pelo IDE, tal como descrito abaixo:

Placa ESP ==> IDE Arduino (gpio)

- D0 ==> 16

- D1 ==> 5
- D2 ==> 4
- D3 ==> 0
- D4 ==> 2
- D5 ==> 14
- D6 ==> 12
- **D7 ==> 13**
- D8 ==> 15
- D9 ==> 3
- D10 ==> 1

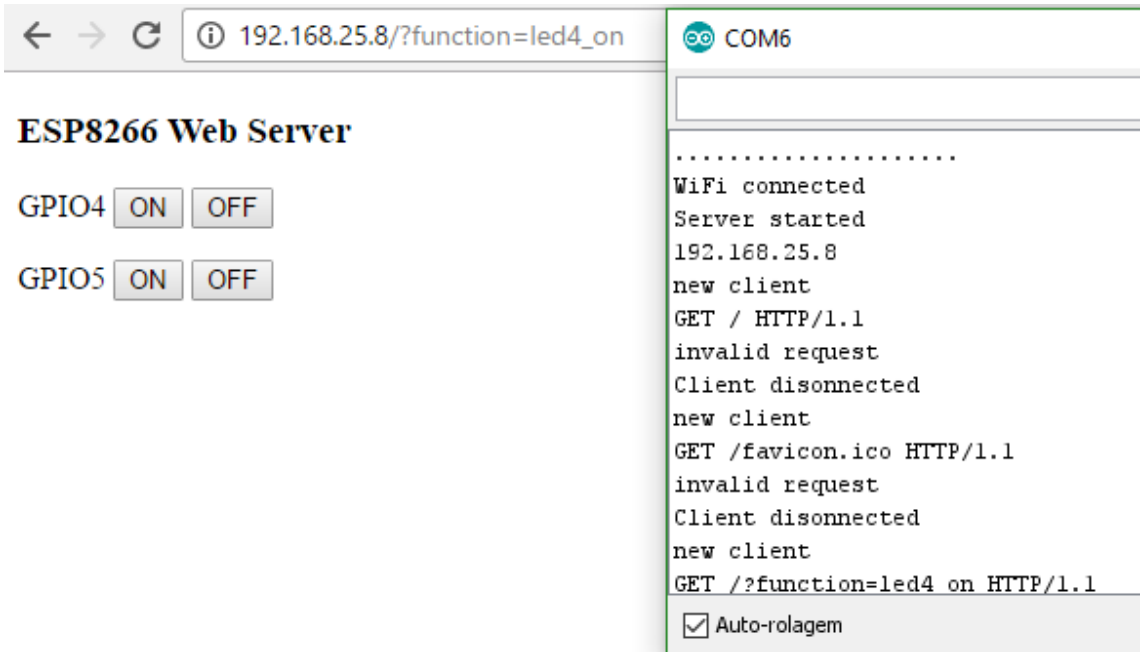
Abaixo o código pronto para ser executado no IDE do Arduino como servidor de página HTML com acionamento dos pinos GPIO 12 e 13:

```
#include <ESP8266WiFi.h>

const char* ssid = "SETA-AF82";
const char* password = "14502384";

WiFiServer server(80);

void setup() {
  Serial.begin(115200);
  delay(10);

  // prepare GPIO 12 => D6 e  GPIO 13 => D7
  pinMode(12, OUTPUT);
  digitalWrite(12, 0);

  pinMode(13, OUTPUT);
  digitalWrite(13, 0);

  // Connect to WiFi network
  Serial.println();
  Serial.println();
  Serial.print("Connecting to ");
  Serial.println(ssid);

  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }

  Serial.println("");
```

```
  Serial.println("WiFi connected");

  // Start the server
  server.begin();
  Serial.println("Server started");
  Serial.println(WiFi.localIP());
}

void loop() {
  WiFiClient client = server.available();
  if (!client) {
    return;
  }

  Serial.println("new client");
  while(!client.available()){
    delay(1);
  }

  String req = client.readStringUntil('\r');
  Serial.println(req);
  client.flush();

  String buf = "";

  buf += "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n<!DOCTYPE HTML>\r\n<html>\r\n";
  buf += "<h3> ESP8266 Web Server</h3>";
  buf += "<p>GPIO12 (D6) <a href=\"?function=led6_on\"><button>ON</button></a> <a
href=\"?function=led6_off\"><button>OFF</button></a></p>";
  buf += "<p>GPIO13 (D7) <a href=\"?function=led7_on\"><button>ON</button></a> <a
href=\"?function=led7_off\"><button>OFF</button></a></p>";
  buf += "<h4>Teste On Off</h4>";
  buf += "</html>\n";

  client.print(buf);
  client.flush();

  if (req.indexOf("led7_on") != -1)
    digitalWrite(13, 1);
  else if (req.indexOf("led7_off") != -1)
    digitalWrite(13, 0);
  else if (req.indexOf("led6_on") != -1)
    digitalWrite(12, 1);
  else if (req.indexOf("led6_off") != -1)
    digitalWrite(12, 0);
  else {
```

```
  Serial.println("invalid request");
  client.stop();
 }
 Serial.println("Client disonnected");
}
```

O *printscreen* abaixo ilustra a operação do código do ESP operando como servidor. Mais detalhes desse exemplo no vídeo em: https://youtu.be/jt_elaBf6YE



Para construir uma página web, embarcar em um servidor e acessá-la a partir de qualquer computador, é possível utilizar a ferramenta online RIB (***https://01.org/rib/online***), que é uma GUI de código aberto para construir páginas web gráficas. Para se tornar funcional, é necessário também adicionar funções JavaScript que abrem o websocket e enviam os dados.